Week 6 Level 3: Advanced

Review and Checkpoint

Connect the pieces, and use the debugger to watch your code think, one line at a time.

What you will learn

- ☐ Use the Thonny debugger to run a program one line at a time and watch the variables.
- ☐ Find and fix bugs in classes, sorts and searches.
- ☐ Show what you know in the mid-level checkpoint.

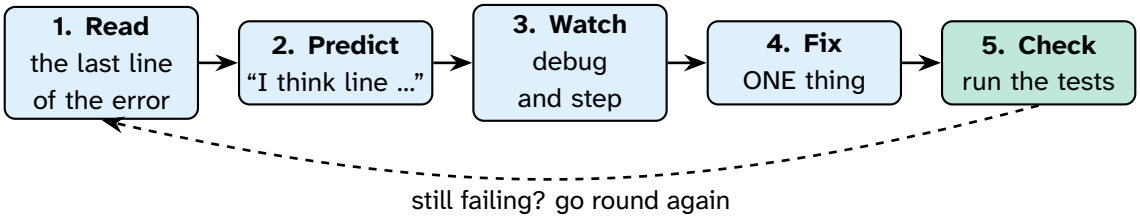
The Thonny debugger

A **debugger** is a slow-motion replay for your code. You can pause on every line and see the value of every variable.

Run	Debug	Step over	Step into	Resume	Stop
runs at full speed	starts slow motion (bug icon)	run this line, stop at the next	go inside a function call	run to the end or a breakpoint	end the program

- Turn on **View > Variables**. Click an object there to see its attributes.
- The highlighted line is the one that will run *next*.
- **Breakpoint:** click a line number to put a red dot on it. Press Debug, then Resume: the program runs and pauses at the dot.

A bug-hunting strategy



Tip: Reading an error

Start at the **bottom**. The last line says *what* went wrong (`TypeError`, `NameError`, `IndexError`…). The line above it says *where*: the file and line number.

Weeks 1–5 in one page

Week	Remember
1 Git	`git add` then `git commit -m "Say what changed"`; `log`, `diff`, `restore`.
2 Objects	`class`, `__init__`, attributes `self.x`, methods `def name(self):`.
3 Families	`class Child(Parent):`, `super().__init__(…)`, overriding; “has a” = composition.
4 Searching	linear: one by one; binary: sorted list, check the middle, halve.
5 Sorting	bubble and selection sort; `sorted(key=...)`; stack (undo) and queue (turns).

Exercise 6.1: Bug Hunt 1: Score Keeper

Folder: `week06/ex-bug-missing-self`. Three bugs, all the same kind. We debug this one together. Write each error and fix:

Exercise 6.2: Bug Hunt 2: Lost Scores

Folder: `week06/ex-bug-lost-items`. A sort that loses scores, and even empties the original list! Watch `scores`, `remaining` and `result` in the Variables view. Three bugs.

Exercise 6.3: Bug Hunt 3: Word Finder

Folder: `week06/ex-bug-binary-search`. Binary search with two off-by-one bugs. First trace the *buggy* code by hand for `find_word(["ant", "bee", "cat"], "ant")`:

low	high	low < high?	mid	words[mid] and so…

Why is “ant” never found? _____

Exercise 6.4: Bug Hunt 4: Shape Party

Folder: `week06/ex-bug-shape-party`. Three inheritance bugs that appear one after another: nothing drawn, then a crash, then diamonds that are only dots.

Vocabulary box

debugger A tool that runs your program in slow motion so you can watch every line and variable.

step over Run the highlighted line in one go, then pause.

step into Go inside the function or method that the line calls.

breakpoint A marked line where the debugger pauses.

traceback The error report Python prints. Read it from the bottom.

trace table A paper table where you write each variable’s value, step by step.

Reflection

My hardest bug today was … and what helped me find it was …

The topic I feel strongest in, and the one I want to practice:

Week 6 Level 3: Advanced

Review and Checkpoint

15–20 minutes. Do Core first; Challenge and Creative are optional.

Name (nickname is fine): _____ Date: _____

★ Core (everyone): Piggy Bank Bug Hunt (about 10 minutes)

1. Open `week06/hw-bug-bank/starter.py`. Read the expected output at the top of the file, then press Run. What is different?
2. Find the 3 bugs. Use the debugger: Debug, Step over, Step into, and watch `self.coins` and `self.log` in the Variables view.
3. Fill in your bug log, then open `check_my_work.py` and press Run until all checks pass. Commit the fixed file in `git-quest`.

Bug	Line	What was wrong	How I fixed it
1			
2			
3			

★★ Challenge (want more?): Trace table

Without running it, fill in the trace table for this code. Then check with the debugger.

```
stack = []
for word in ["red", "green", "blue"]:
    stack.append(word)
top = stack.pop()
```

After line	stack	top
append “red”		
append “green”		
append “blue”		
`top = stack.pop()`		

Creative (make it yours): Bug comic

Draw a 3-panel comic about a bug you met this term: panel 1, the bug appears; panel 2, the hunt (what strategy did you use?); panel 3, the fix. Bugs can be cute!

Self-check: tick each box when it is true

- ☐ All checks pass in `hw-bug-bank`.
- ☐ I used the debugger at least once at home.
- ☐ I changed one thing at a time and ran the program after each change.
- ☐ I committed my fixed file in `git-quest`.